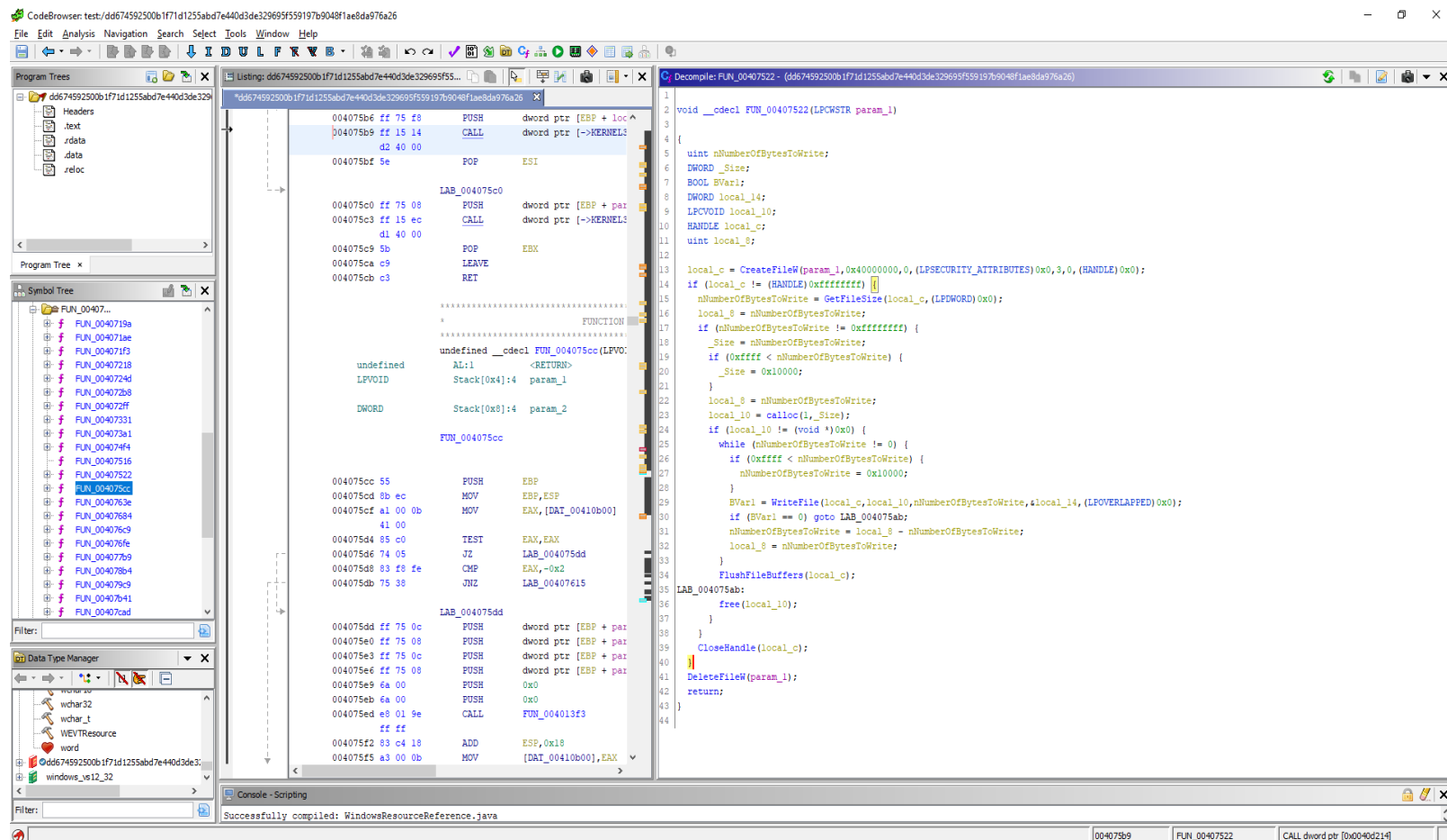




GBA + RE

Ivan Parkhomchyk Patapchyk
Abril 2026



¿Por qué?
¿Y por qué no?

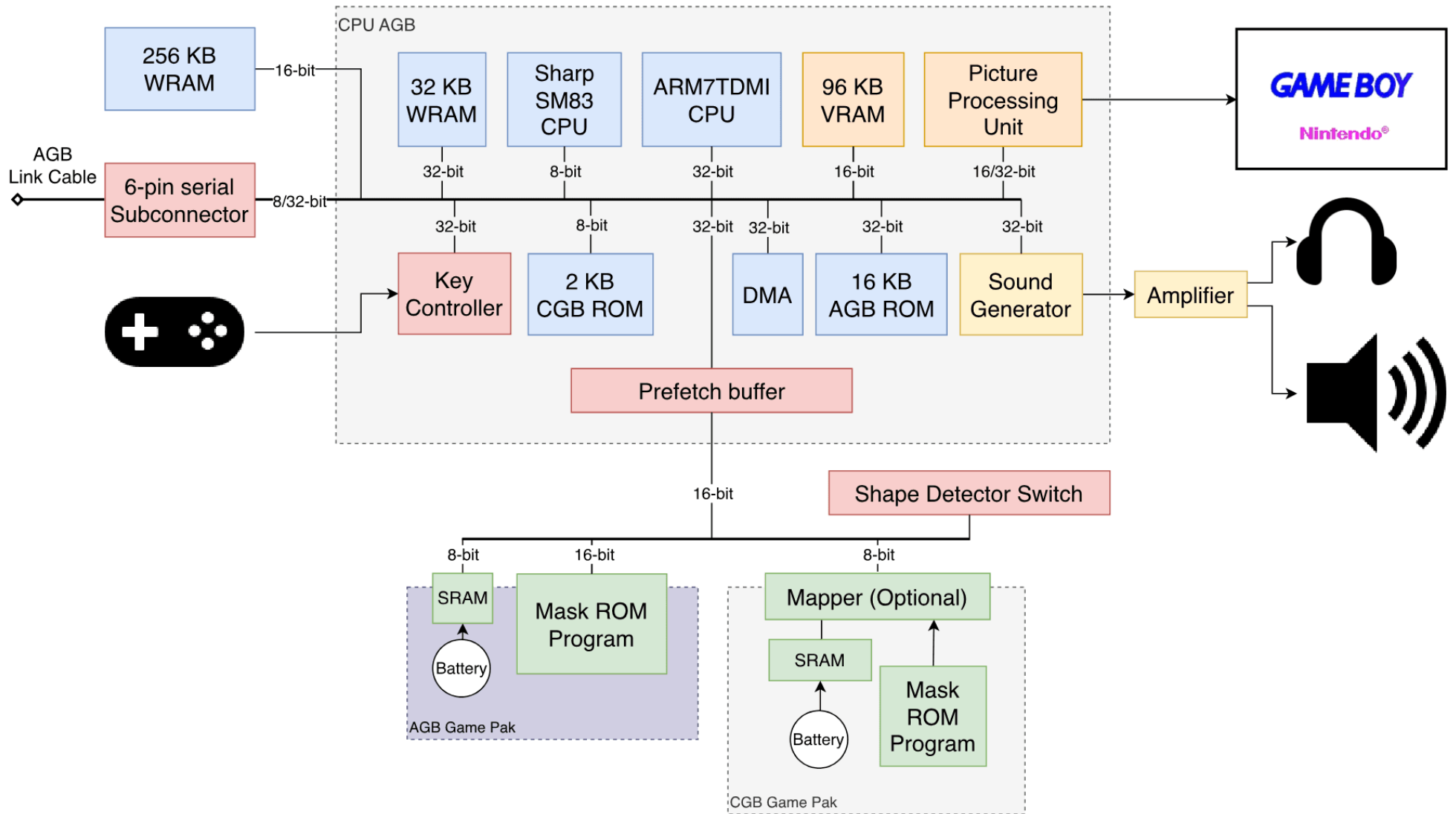
GBA

Características de la GBA

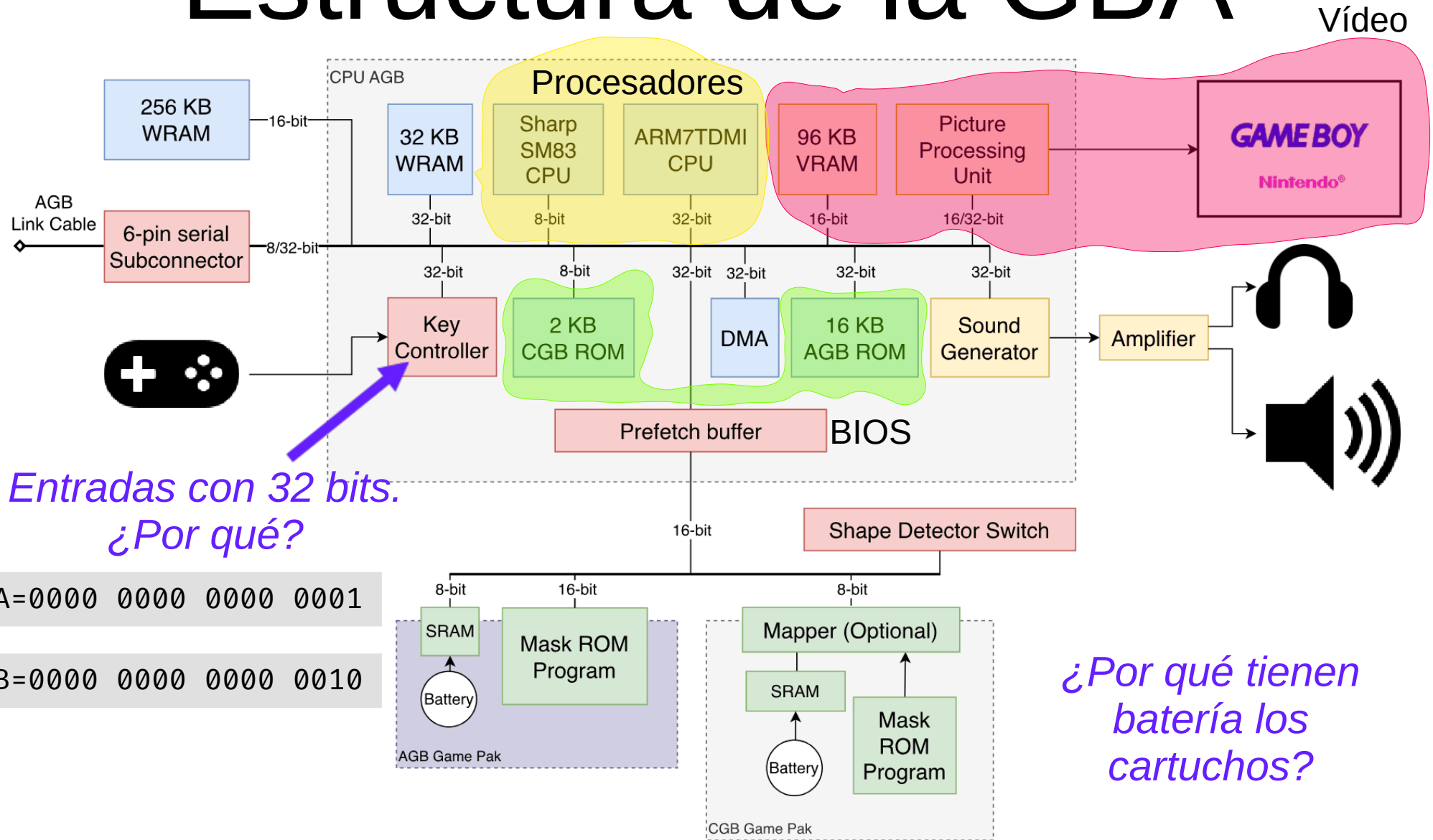


- 2001
- 32 bit
- 3D → transformaciones afines
- 2D → colores, paletas
- Pantalla rectangular → mayor resolución
- Nuevos botones → L, R
- 2 CPUs
- Sucesora a Game Boy Color

Estructura de la GBA



Estructura de la GBA



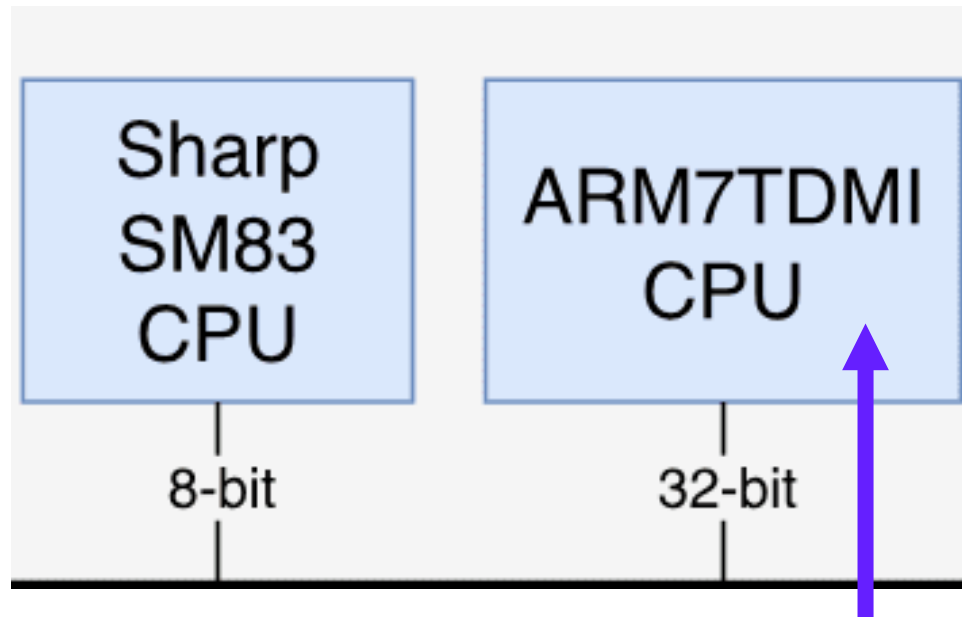
¿1 console, 2 cpus?

*!!!La respuesta está en la **retrocompatibilidad**!!!*

- La GBA trabaja con una CPU a la vez.
- Dada su capacidad de ejecutar GB o GBA,
 - → en función de la ROM, se activa una u otra.

Game Boy (CGB)
(generación anterior)

Game Boy Advance (AGB)
(generación GBA)

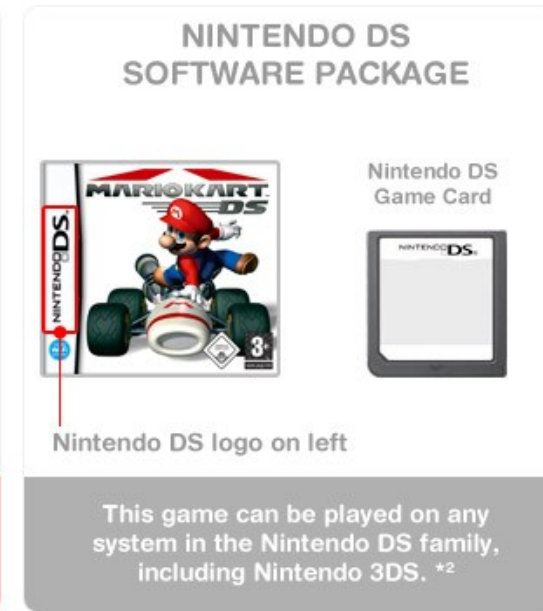


Nos enfocaremos
en este segundo

Compatibilidad

- Nintendo en sus videoconsolas ha incluido múltiples CPUs a lo largo de múltiples generaciones.
- Patrón muy socorrido para garantizar la máxima retrocompatibilidad con la generación anterior (si la emulación resulta en estado muy verde).
- Ejemplo: **DS, 3DS y GBA**
- **De hecho en las videoconsolas mencionadas, se usan a la vez.**

❌ ¡Esto es un bulo!



SOFTWARE COMPATIBILITY CHART

	Nintendo 3DS	Nintendo DSi XL	Nintendo DSi	Nintendo DS Lite	Nintendo DS
Nintendo 3DS Software	✓	✗	✗	✗	✗
Nintendo DSiWare	✓ *1	✓	✓	✗	✗
Nintendo DSi Software	✓ *1	✓	✓	✗	✗
Nintendo DS Software	✓ *1, 2	✓ *2	✓ *2	✓	✓
Game Boy Advance Software	✗ *2	✗ *2	✗ *2	✓	✓

ARM7TDMI

(Rev 3)

Technical Reference Manual

ARM

ARMv7 TDMI

- Procesador usado para ROMs GBA.
- RISC.
- Instrucciones de tamaño fijo.
- Ejecución condicional.
- 32 bits.
- Dos modalidades de instrucciones.
- Más registros.
- Depuración JTAG.

Cauce con adelantamiento → NOPs

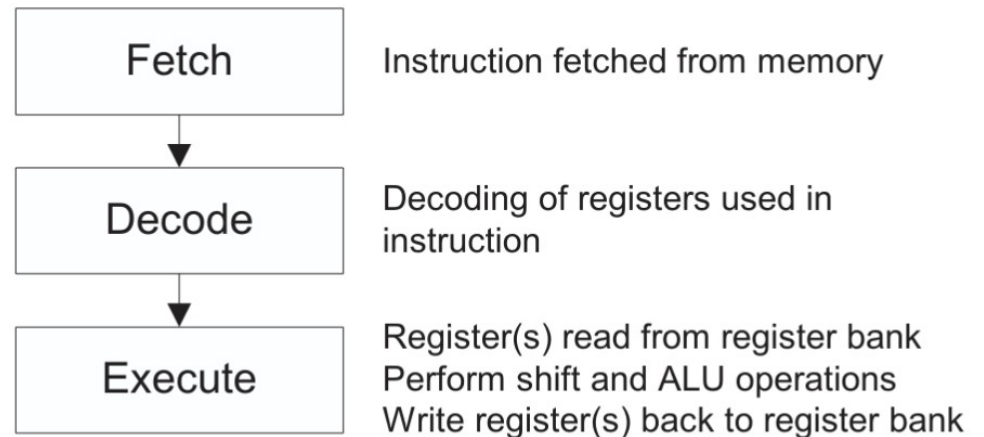


Figure 1-1 Instruction pipeline

Instruc. condicionales

Riesgos de control controlados con Cond

		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Data processing and FSR transfer	Cond	0	0	1	Opcode				S	Rn				Rd				Operand 2																	
Multiply	Cond	0	0	0	0	0	0	A	S	Rd				Rn				Rs				1	0	0	1	Rm									
Multiply long	Cond	0	0	0	0	1	U	A	S	RdHi				RdLo				Rn				1	0	0	1	Rm									
Single data swap	Cond	0	0	0	1	0	B	0	0	Rn				Rd				0	0	0	0	1	0	0	1	Rm									
Branch and exchange	Cond	0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	1	Rn					
Halfword data transfer, register offset	Cond	0	0	0	P	U	0	W	L	Rn				Rd				0	0	0	0	1	S	H	1	Rm									
Halfword data transfer, immediate offset	Cond	0	0	0	P	U	1	W	L	Rn				Rd				Offset				1	S	H	1	Offset									
Single data transfer	Cond	0	1	1	P	U	B	W	L	Rn				Rd				Offset																	
Undefined	Cond	0	1	1																					1										
Block data transfer	Cond	1	0	0	P	U	S	W	L	Rn				Register list																					
Branch	Cond	1	0	1	L	Offset																													
Coprocessor data transfer	Cond	1	1	0	P	U	N	W	L	Rn				CRd				CP#				Offset													
Coprocessor data operation	Cond	1	1	1	0	CP Opc				CRn				CRd				CP#				CP		0	CRm										
Coprocessor register transfer	Cond	1	1	1	0	CP Opc				L	CRn				Rd				CP#				CP		1	CRm									
Software interrupt	Cond	1	1	1	1	Ignored by processor																													
		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		

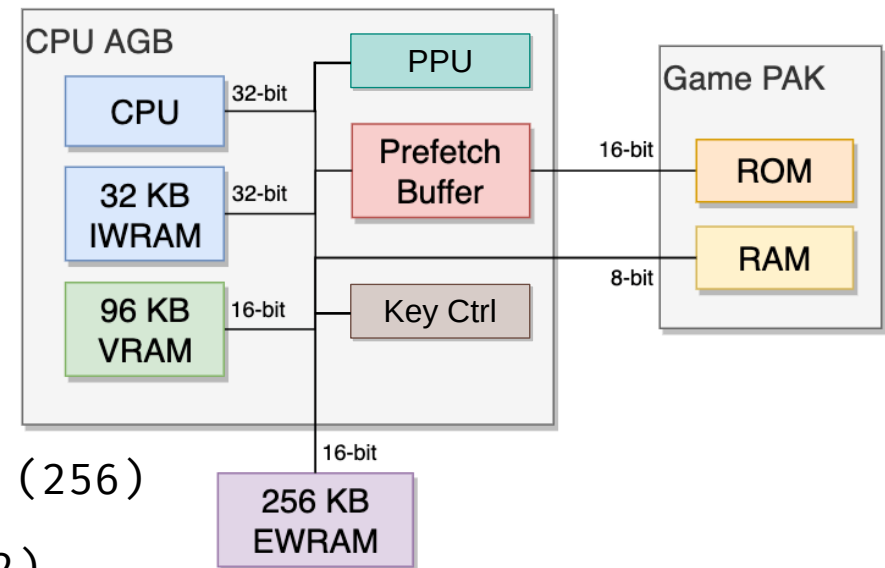
Thumb

- 16 bits.
- Ocupa menos espacio en el bus.
- Menos instrucciones.
- Menos registros.
- Más rápido.
- No hay ejecución condicional.
- Subconjunto de instrucciones ARM.

Podemos intercambiar
entre ARM y Thumb
durante la ejecución

	Format	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Move shifted register	01	0	0	0	Op	Offset5					Rs			Rd				
Add and subtract	02	0	0	0	1	1	1	Op	Rn/ offset3			Rs			Rd			
Move, compare, add, and subtract immediate	03	0	0	1	Op	Rd			Offset8									
ALU operation	04	0	1	0	0	0	0	Op			Rs			Rd				
High register operations and branch exchange	05	0	1	0	0	0	1	Op	H1	H2	Rs/Hs			RdHd				
PC-relative load	06	0	1	0	0	1	Rd			Word8								
Load and store with relative offset	07	0	1	0	1	L	B	0	Ro			Rb			Rd			
Load and store sign-extended byte and halfword	08	0	1	0	1	H	S	1	Ro			Rb			Rd			
Load and store with immediate offset	09	0	1	1	B	L	Offset5					Rb			Rd			
Load and store halfword	10	1	0	0	0	L	Offset5					Rb			Rd			
SP-relative load and store	11	1	0	0	1	L	Rd			Word8								
Load address	12	1	0	1	0	SP	Rd			Word8								
Add offset to stack pointer	13	1	0	1	1	0	0	0	0	S	SWord7							
Push and pop registers	14	1	0	1	1	L	1	0	R	Rlist								
Multiple load and store	15	1	1	0	0	L	Rb			Rlist								
Conditional branch	16	1	1	0	1	Cond					Softset8							
Software interrupt	17	1	1	0	1	1	1	1	1	Value8								
Unconditional branch	18	1	1	1	0	0	Offset11											
Long branch with link	19	1	1	1	1	H	Offset											
	Format	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	

Mapa memoria



00000000-00003FFF

BIOS (16)

00004000-01FFFFFF

02000000-0203FFFF

EWRAM | RAM cartucho (256)

02040000-02FFFFFF

03000000-03007FFF

IWRAM | RAM placa (32)

03008000-03FFFFFF

04000000-040003FE

IO RAM | Registros de I/O

04000400-04FFFFFF

05000000-050003FF

PAL RAM | Paletas (1)

05000400-05FFFFFF

06000000-06017FFF

VRAM | Video RAM (96)

06018000-06FFFFFF

07000000-070003FF

OAM | Sprites (1)

07000400-07FFFFFF

08000000-0DFFFFFF

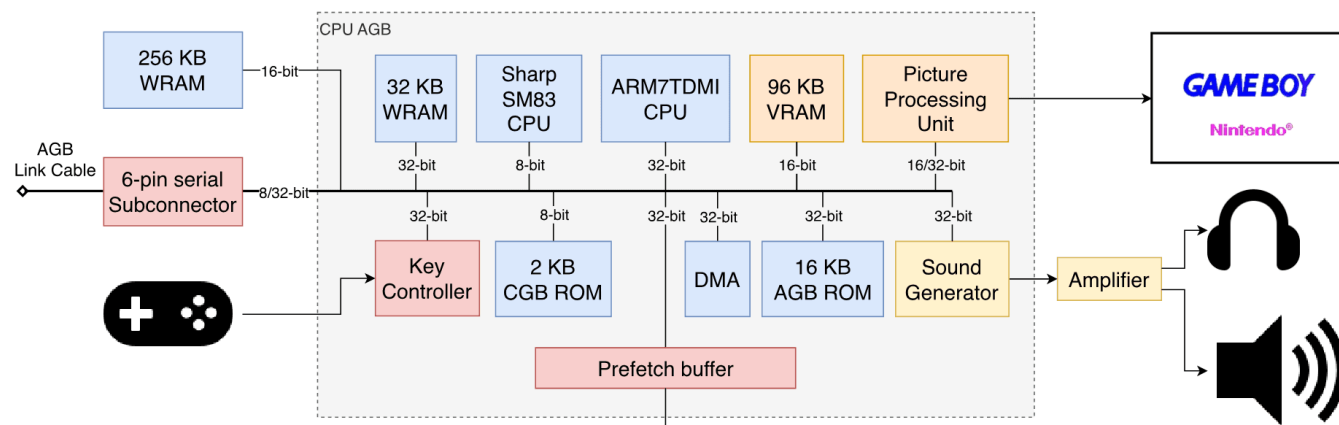
PAK ROM | Game Pak ROM/FlashROM (max 32MB)

0E000000-0E00FFFF

Card RAM | Game Pak SRAM (max 64 KBytes)

0E010000-0FFFFFFF

10000000-FFFFFFFF

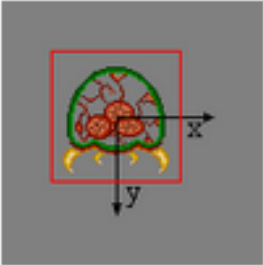
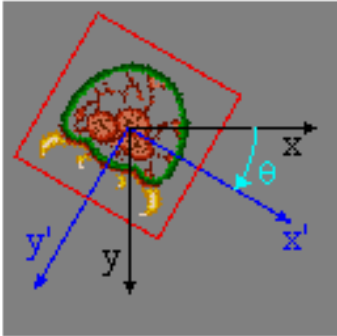
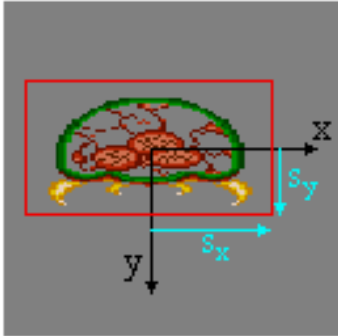
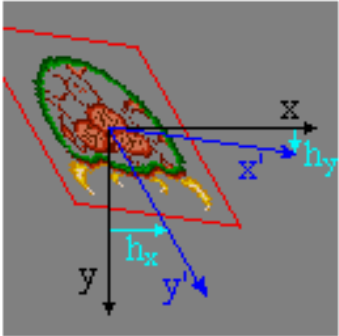


Vídeo

- Pantalla LCD de 240 x 160 px a ~60 FPS.
- Componentes involucrados: OAM, PAL RAM, VRAM y PPU.
 - La GBA puede dibujar en “3D”. Iteración del modo 7.
 - Transformaciones afines.

Transformación afín

Table 10.1: transformation matrices and their inverses.

Identity	Rotation	Scaling	Shear
			
$\mathbf{I} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\mathbf{R}(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$	$\mathbf{S}(s_x, s_y) = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$	$\mathbf{H}(h_x, h_y) = \begin{bmatrix} 1 & h_x \\ h_y & 1 \end{bmatrix}$
$\mathbf{I}^{-1} = \mathbf{I}$	$\mathbf{R}^{-1}(\theta) = \mathbf{R}(-\theta)$	$\mathbf{S}^{-1}(s_x, s_y) = \mathbf{S}\left(\frac{1}{s_x}, \frac{1}{s_y}\right)$	$\mathbf{H}^{-1}(h_x, h_y) = \frac{\mathbf{H}(-h_x, -h_y)}{1 - h_x h_y}$

We can now use these definitions to find the correct matrix for enlargements by s_x and s_y , followed by a **counter-clockwise** rotation by α ($=-\theta$), by matrix multiplication.

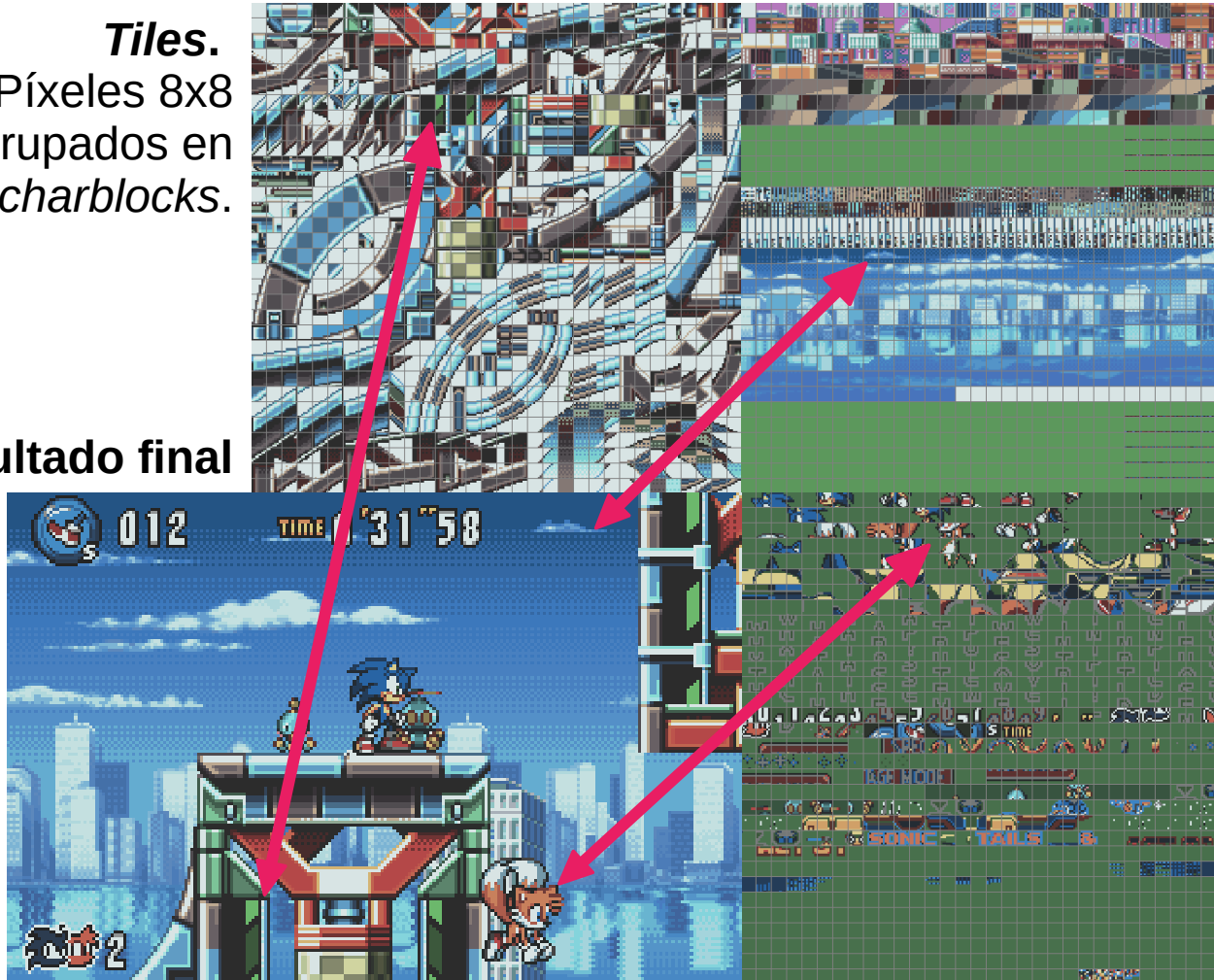
Gráficos 2D

Los modos 0, 1 y 2 → 2D con algunas limitaciones y difirencias.

Tiles.

- Píxeles 8x8
- Agrupados en *charblocks*.

Resultado final



Fondos.

- Hasta 4 capas.
- Agrupados en *tile maps*.
- Formados por *screenblocks* de 32x32 *tiles*.

Sprites.

- “Lo móvil”.
- Hasta 64x64 px.
- Transformaciones afines:
 - Escalado
 - Rotación

¿En qué modo de vídeo 2D estamos en esta pantalla?
Nota: no se produce ninguna transformación en el gameplay.

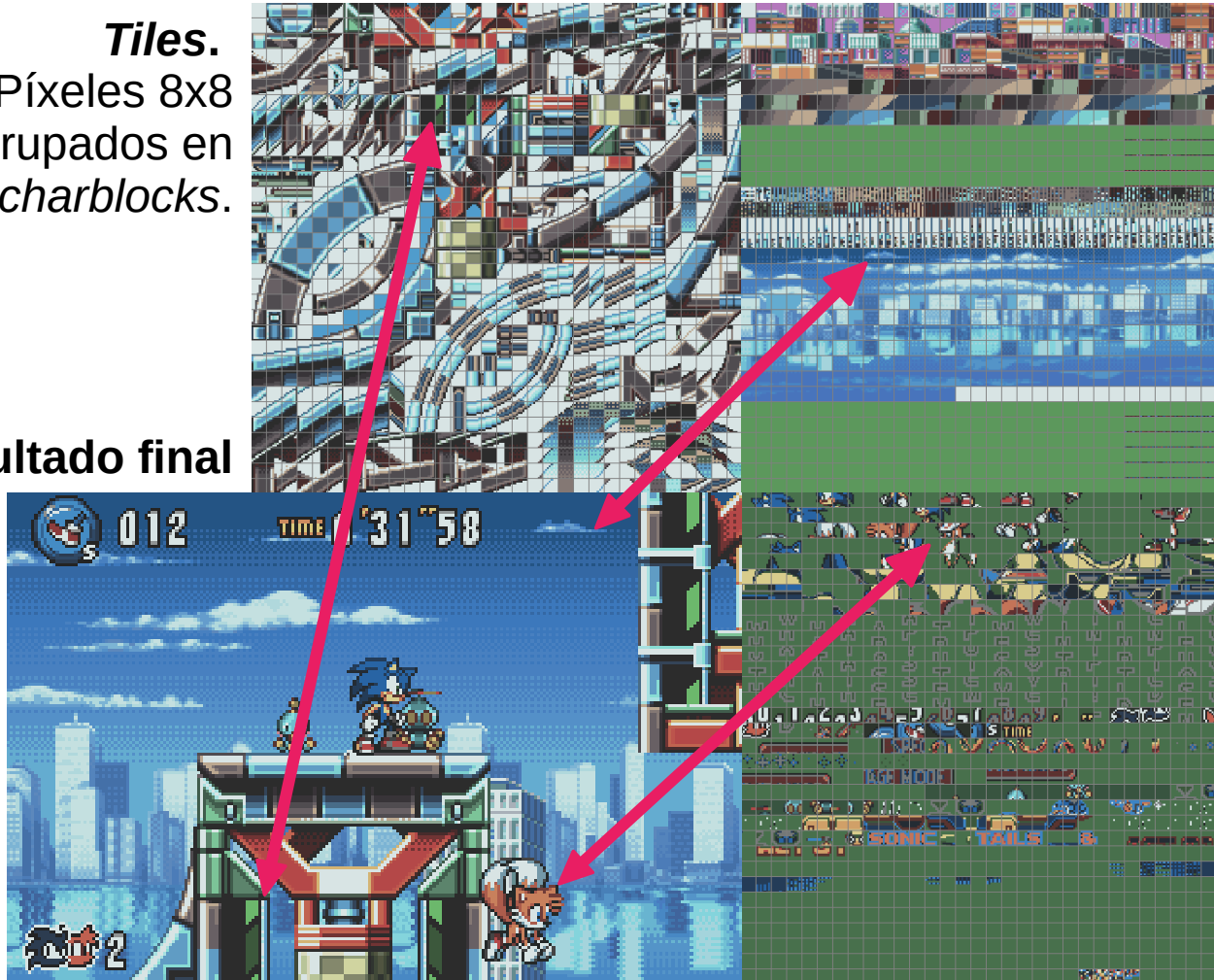
Gráficos 2D

Los modos 0, 1 y 2 → 2D con algunas limitaciones y difirencias.

Tiles.

- Píxeles 8x8
- Agrupados en *charblocks*.

Resultado final



Fondos.

- Hasta 4 capas.
- Agrupados en *tile maps*.
- Formados por *screenblocks* de 32x32 *tiles*.

Sprites.

- “Lo móvil”.
- Hasta 64x64 px.
- Transformaciones afines:
 - Escalado
 - Rotación

¿En qué modo de vídeo 2D estamos en esta pantalla?

Nota: no se produce ninguna transformación en el gameplay.

Modo 0 → 4 capas (con scroll) → NO HAY TRANSFORMACIÓN AFÍN

Gráficos 3D

¿Cuál *gameplay* es 3D real?

- Mapa de bits.
- Limitaciones de *sprites*.
- Mucha CPU
 - Consumo.
- Modos 3, 4 y 5.



Gráficos 3D

Fondo scroll.

Carretera plana.
Capa afín.

Sprite.
Píxel-perfecto

Terreno
poligonal

Rota 3D el quad.
NO Píxel-perfecto



No trataremos...



Entrada



Homebrew y sistemas
antipiratería



Sistemas de
guardado



Sonido



Otros periféricos

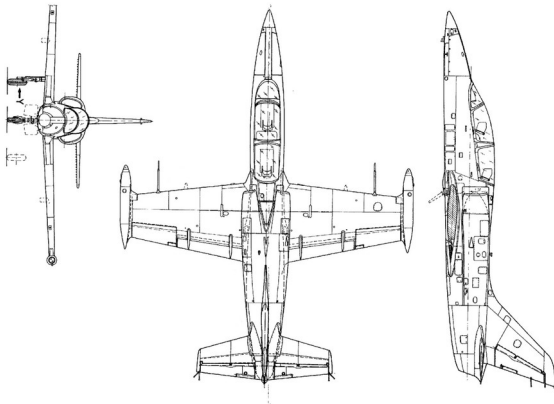


... por falta de tiempo

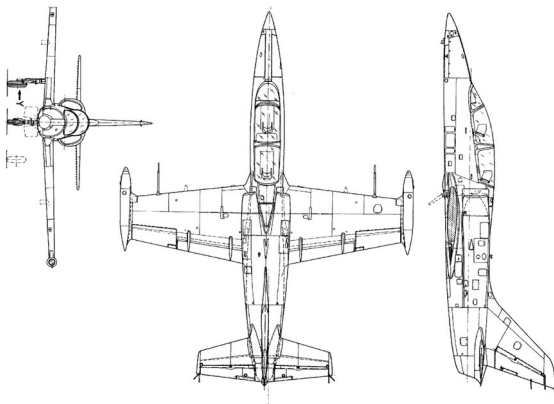
RE

Reverse Engineering

Ingeniería directa



Ingeniería inversa (RE)



Ej. clásico informática

Sistemas Unix
GNU/Linux, Mac, iOS, Solaris, etc



SAMBA

SMB/CIFS



Carpeta compartida

Sistemas Windows
Windows 98, XP, 7, 10, Server 2021, etc



SMB/CIFS

SAMBA

Sistemas Unix
GNU/Linux, Mac, iOS, Solaris, etc

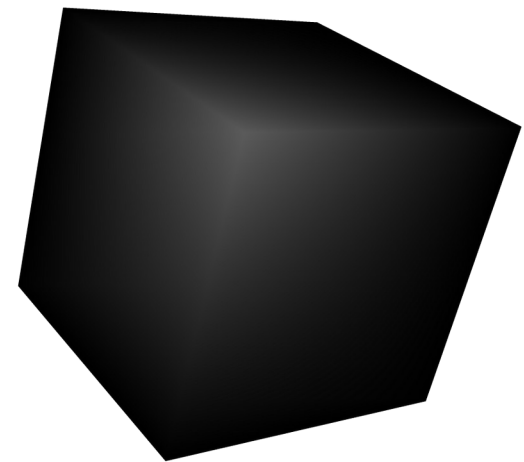


Impresora compartida

¿Por qué es necesario?

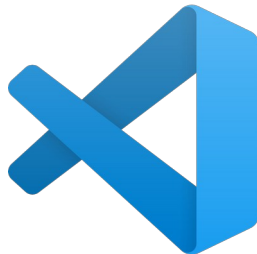
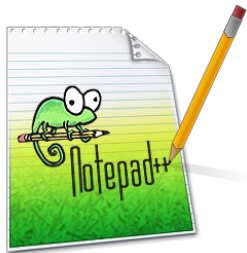
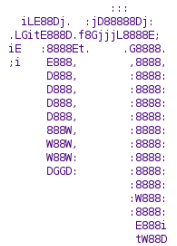
- Seguridad e integridad
- Copias y preservación
- Detección de violación de licencias
- Competencias

*El software es tratado
como una caja negra*



Programas: editor texto

- Escribir código cómodamente
- Extensiones



- Visualizar y editar binarios.

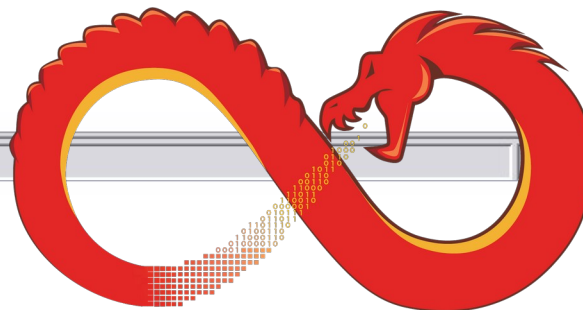
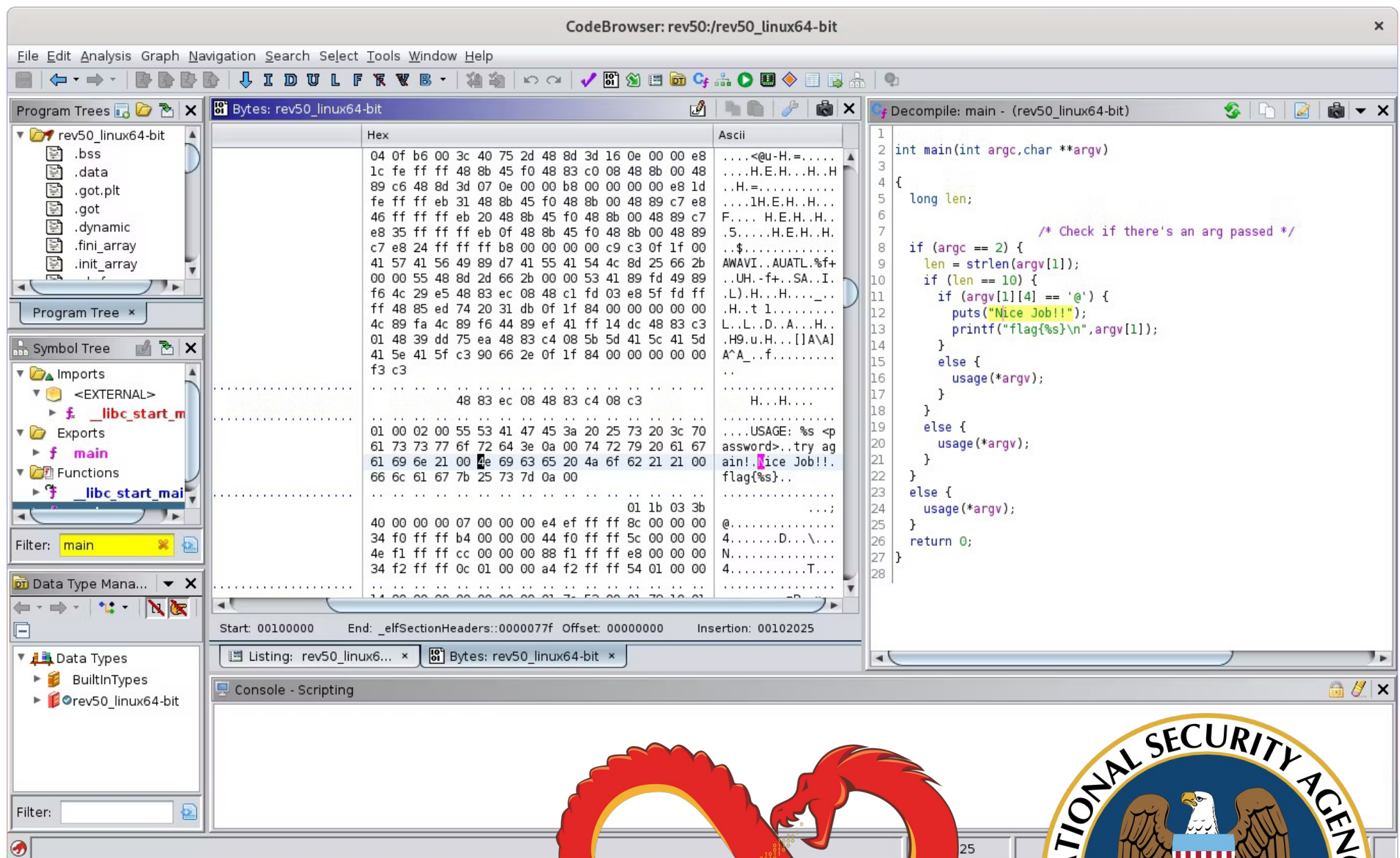


Programas: compilador y depurador

- Depende mucho del *stack*.
 - No es lo mismo Web que trabajar con ensamblados de C#.
- Opciones típicas: edb, gdb, gcc, clang, llvm...
- depurador → fundamental

Programas: desensamblador y decompilador

- *Disassembler* → traductor.
- *Decompiler* → aplica heurísticas para conseguir un posible código fuente.
- Programas esenciales.
- Depende del *stack* tecnológico.



GHIDRA

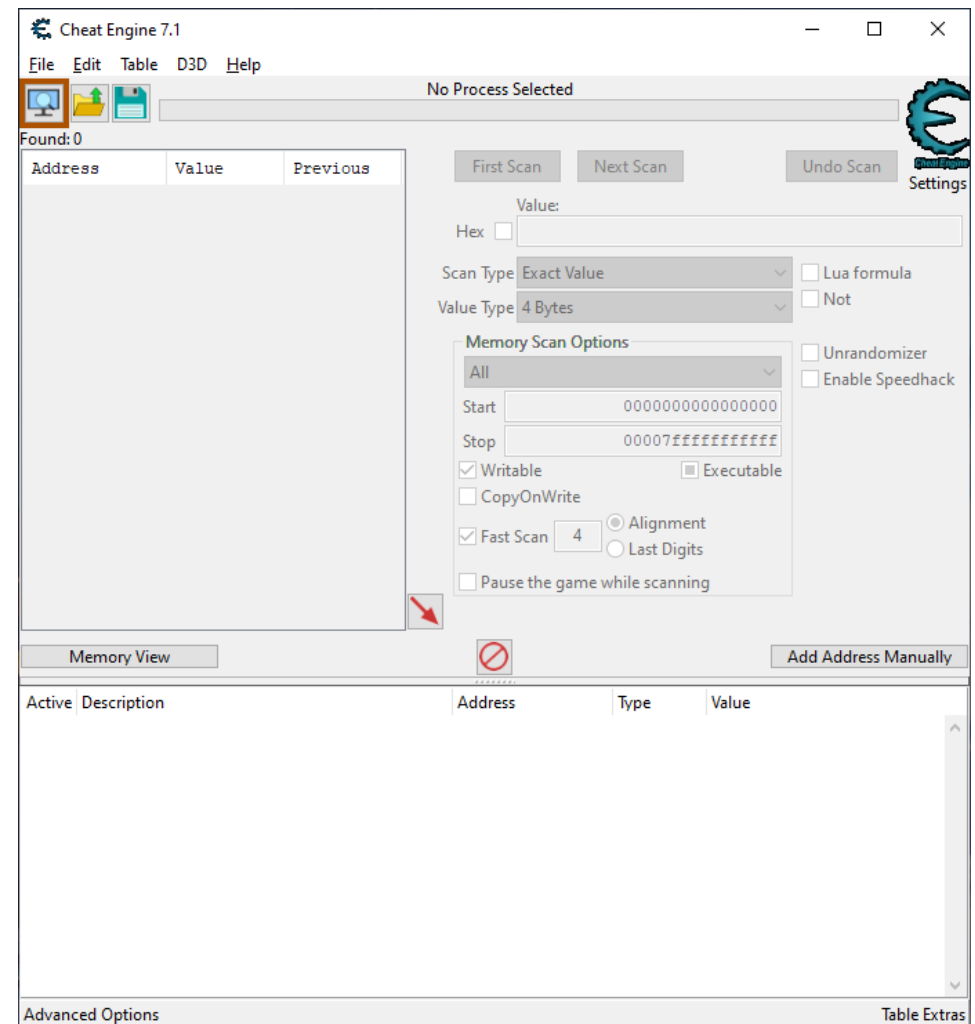


```
testcases — bash — 121x20
stimpy:$java -jar cfr_0_130.jar org/codehaus/plexus/configuration/xml/XmlPlexusConfiguration.class --methodname toString
public String toString() {
    StringBuilder buf = new StringBuilder().append('<').append(this.getName());
    for (String a : this.getAttributeNames()) {
        buf.append(' ').append(a).append("=\").append(this.getAttribute(a)).append('\");
    }
    if (this.getChildCount() > 0) {
        buf.append('>');
        for (PlexusConfiguration c : this.getChildren()) {
            buf.append(c);
        }
        buf.append("</").append(this.getName()).append('>');
    } else if (null != this.getValue()) {
        buf.append('>').append(this.getValue()).append("</").append(this.getName()).append('>');
    } else {
        buf.append(">");
    }
    return buf.append('\n').toString();
}
stimpy:$
```



Programas: Escáner memoria

- Visualización y alteración de la memoria de un programa.
 - Trampas.
 - Ubicar variables.



Programas: aislamiento

- Seguridad: entornos aislados o desechables.
- No sobre nuestra máquina si ello implica la ejecución de código malicioso o poco probado.
- Problemas de compatibilidad, entorno no real...

Programas: aislamiento



- **Emulación.**

- GBA → x86

- **Virtualización.**

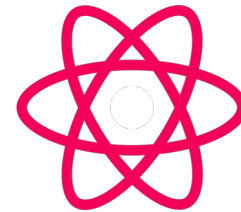
- HaikuOS en Windows.

- Misma arquitectura.



Programas: aislamiento

- **Capa de compatibilidad.**
- Traducción de instrucciones binarias.
- GTA VI → linux.
- Intel mac → silicon mac



GloriousEggroll/
ULWGL

Unified Linux Wine Game Launcher

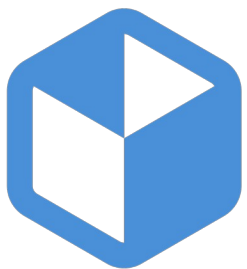


2
Contributors

1
Issue

335
Stars

3
Forks



Flatpak



docker



Waydroid



podman

- **Contenerización.**
- Despliegue → Linux ligero.
- Permisos. *Sandbox.*

Taller práctico

GBA + RE

Objetivos

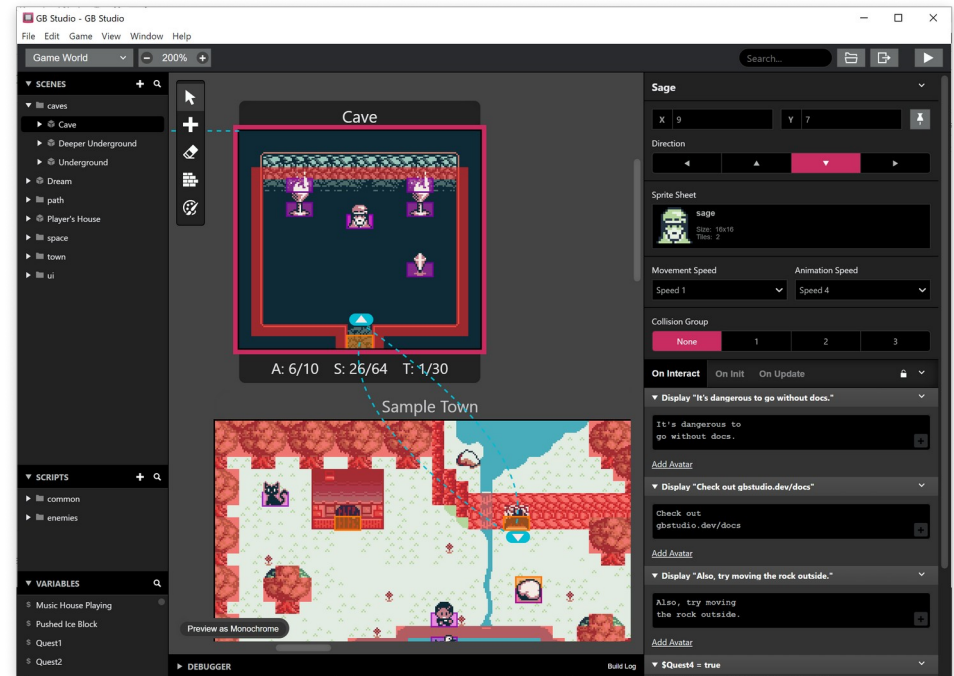
- Crear SW *homebrew* para GBA.
- *¿Qué es esto?*
Depurar y experimentar con la ROM ejecutable.
- Inspeccionar y alterar el binario.
- Familiarizarse con el *tooling*.
- Reflejar y valorar el trabajo de un emulador y herramientas.

¿Qué vamos a gastar?

- Un **SDK** de código abierto para compilar y depurar.



- ¿Qué otros hay?
- *Unix-like* (en windows es posible).
- MGBA → emulador



¿Qué vamos a gastar?

- **Docker (rápido)** o sin docker (a mano: pacman).
- En la guía, el editor usado es . Pero podéis utilizar el que queráis.
 - Algunas configuraciones son opcionales pero adaptarlas al editor es cosa de cada uno.

Preparación del entorno

Práctica 0

Pasos muy guiados

- Instalando devKitPro. Comprobación.
 - Si tienes problemas con docker...
- Instalando *tooling*. Comprobación.
 - Estamos instalando appimages.
 - “Ejecutables portables” para gnu/linux.
- Copiando ejemplos al directorio.

Lee con
atención

No olvides ver el siguiente paso.

Paso 1: **Dockerfile**

Paso 2: Construcción de imagen

Paso 3: Comprobación



Para facilitar la instalación devKitPro, emplearemos una imagen de docker.

ES OPCIONAL
CONFIGURAR EL
CLANGD

NO HACER SI HAY POCO
TIEMPO



Compilando de ROM básica. Ejecución emulador.

Práctica 1

Analicemos el Makefile

- Ve a `ejemplos_gba/template`.
- Abre el Makefile y contesta.
 - Tienes una copia del makefile en la wiki.
- ¿Qué regla nos genera el binario final?
- ¿Podemos utilizar C++? Si es así, ¿Qué regla emplea compilador que toca?
- Observa que se emplea la siguiente orden `ARCH := -mthumb -mthumb-interwork`, ¿esto quiere decir que el programa generado solo contendrá instrucciones Thumb?

¿Qué regla nos genera el binario final?

```
#-----  
# main targets  
#-----  
  
$(OUTPUT).gba : $(OUTPUT).elf  
$(OUTPUT).elf : $(OFILES)  
$(OFILES_SOURCES) : $(HFILES)
```

```
graph LR; elf["$(OUTPUT).elf"] --> gba["$(OUTPUT).gba"]; elf --> ofiles["$(OFILES)"]
```

¿Podemos utilizar C++? Si es así, ¿Qué regla emplea compilador que toca?

Sí, si podemos

```
CPPFILES      := $(foreach dir,$(SOURCES),$(notdir $(wildcard $(dir)/*.cpp)))

# ...

#-----
# use CXX for linking C++ projects, CC for standard C
#-----
ifeq ($(strip $(CPPFILES)),)
#-----
    export LD      := $(CC)
#-----
else
#-----
    export LD      := $(CXX)
#-----
endif
#-----
```


Observa que se emplea la siguiente orden
ARCH := -mthumb -mthumb-interwork,
¿esto quiere decir que el programa generado solo
contendrá instrucciones Thumb?

No

*Si consultas el manual del gcc, verás que explícitamente no hace eso.
Además, concuerda con lo dicho en la primera parte de la wiki.*

¡compilar y abre el emulador!

template.c

```
#include <gba_console.h>
#include <gba_video.h>
#include <gba_interrupt.h>
#include <gba_systemcalls.h>
#include <gba_input.h>
#include <stdio.h>
#include <stdlib.h>
```

← *Bibliotecas del devkitpro*

```
// _____
// Program entry point
// _____
int main(void) {
// _____
```

```
    // the vblank interrupt must be enabled for VBlankIntrWait() to work
    // since the default dispatcher handles the bios flags no vblank handler
    // is required
```

```
    irqInit();
    irqEnable(IRQ_VBLANK);
```

← *Dibujar a ~60 fps*

```
    consoleDemoInit();
```

← *Esto pone Modo 0 y carga una fuente como sprite. GBA NO TIENE std::cout real.*

```
    // ansi escape sequence to set print co-ordinates
    // /x1b[line;columnH
```

```
    iprintf("\x1b[10;10HHello World!\n");
```

```
    while (1) {
        VBlankIntrWait();
    }
```

← *Coloca en 10, 10 esa frase. Fíjate es iprintf.*

```
}
```

← *Esperar al fotograma → ahorro de energía*

Alterando *tiles*...

The image shows a Windows desktop with the mGBA emulator running. Three windows are open: 'Visor de memoria' (Memory Viewer), 'Tiles' (Tile Editor), and the mGBA project window. Red arrows and numbers (1-12) indicate a workflow for altering tiles.

Visor de memoria (Memory Viewer):

- 5: Dropdown menu set to 'Video RAM (96KiB)'.
- 6: Radio button selected for '4 Bytes' inspection direction.
- 7: Address '06000900' highlighted in the VRAM list.
- 8: Value '00000000' highlighted in the data column.
- 9: 'Entero sin signo' (Unsigned Integer) field set to '0'.
- 10: Address '06000900' highlighted in the data column.
- 11: 'Entero sin signo' (Unsigned Integer) field set to '0'.

Tiles (Tile Editor):

- 1: 'Herramientas' (Tools) menu.
- 2: 'Dirección' (Address) field set to '0x06000900'.
- 3: 'Dirección' (Address) field set to '0x06000900'.
- 4: 'Herramientas' (Tools) menu.
- 5: 'Paleta' (Palette) dropdown set to '0'.
- 6: '256 colores' (256 colors) option.
- 7: 'Ampliación' (Zoom) dropdown set to '1x'.
- 8: 'Tiles por fila' (Tiles per row) dropdown set to '32'.
- 9: 'Ajustar a ventana' (Fit to window) checkbox checked.
- 10: 'Displayed tiles' section with 'Ambos' (Both) selected.
- 11: 'Copiar seleccionados' (Copy selected) button.
- 12: 'Copiar todos' (Copy all) button.

mGBA - project (60.1 fps) - 0.10.5:

- 1: 'Herramientas' (Tools) menu.
- 2: 'Herramientas' (Tools) menu.
- 3: 'Herramientas' (Tools) menu.
- 4: 'Herramientas' (Tools) menu.
- 5: 'Herramientas' (Tools) menu.
- 6: 'Herramientas' (Tools) menu.
- 7: 'Herramientas' (Tools) menu.
- 8: 'Herramientas' (Tools) menu.
- 9: 'Herramientas' (Tools) menu.
- 10: 'Herramientas' (Tools) menu.
- 11: 'Herramientas' (Tools) menu.
- 12: 'Herramientas' (Tools) menu.

The game screen displays 'Hello World!' in a pixelated font.

Si hay tiempo...

- Investigad la cabecera `gba_input.h`, y modificad el programa para poder mover el texto con la cruceta.
- **Recuerda** cómo se representa la entrada y **cuidado** al manejar el bucle infinito.



Solución propuesta

```
#include ...  
  
int  
main (void) {  
    int x = 10, y = 10;  
    int old_x = x, old_y = y;  
    u16 input;
```

```
    irqInit ();  
    irqEnable (IRQ_VBLANK);
```

```
    consoleDemoInit ();  
    iprintf ("\x1b[%d;%dHHello World!", y, x);
```

```
    while (1) {
```

```
        scanKeys ();  
        input = keysHeld ();
```

```
        old_x = x;  
        old_y = y;
```

```
        if (input & KEY_DOWN) ++y;  
        if (input & KEY_UP) --y;  
        if (input & KEY_RIGHT) ++x;  
        if (input & KEY_LEFT) --x;
```

```
        if (x < 0) x = 0;  
        if (x > 18) x = 18;  
        if (y < 0) y = 0;  
        if (y > 19) y = 19;
```

```
        if (x != old_x || y != old_y) {  
            iprintf ("\x1b[%d;%dH", old_y, old_x);  
            iprintf ("\x1b[%d;%dHHello World!", y, x);  
        }
```

```
        VBlankIntrWait ();
```

```
    }  
}
```

Variable global de entrada ¿Por qué es de 16 bits (u16) si dijimos que era de 32 bits?

Copiar registros de entrada al programa

Lógica de movimiento

Tratar desbordamiento del texto de fuera de la pantalla.

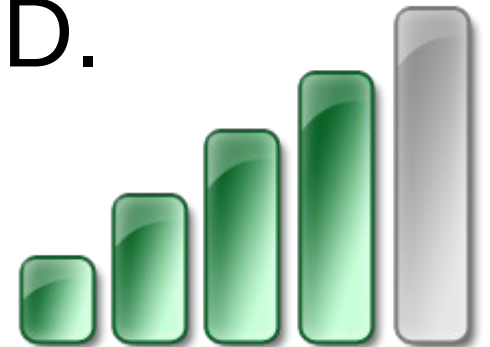
Limpiar lo anterior y dibujar lo nuevo.

Depuración remota. Utilidades++. Lvl. instrucción.

Práctica 2

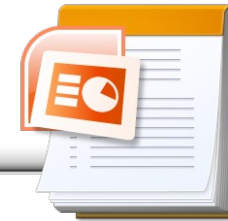
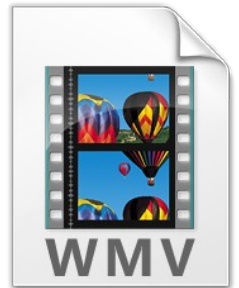
¿Qué vamos a hacer?

- Alterar el binario.
 - **Compilad** el código.
 - Aunque esto para que sea realmente RE, vamos a suponer que nos hemos olvidado del código fuente :D.
- Por falta de tiempo, no depuraremos
 - Pero contaremos sobre cómo funciona la depuración remota.



Herramienta file

- Nos dice qué tipo de archivo es.
- buscan de patrones y números mágicos



shell - GNU Emacs at fedora

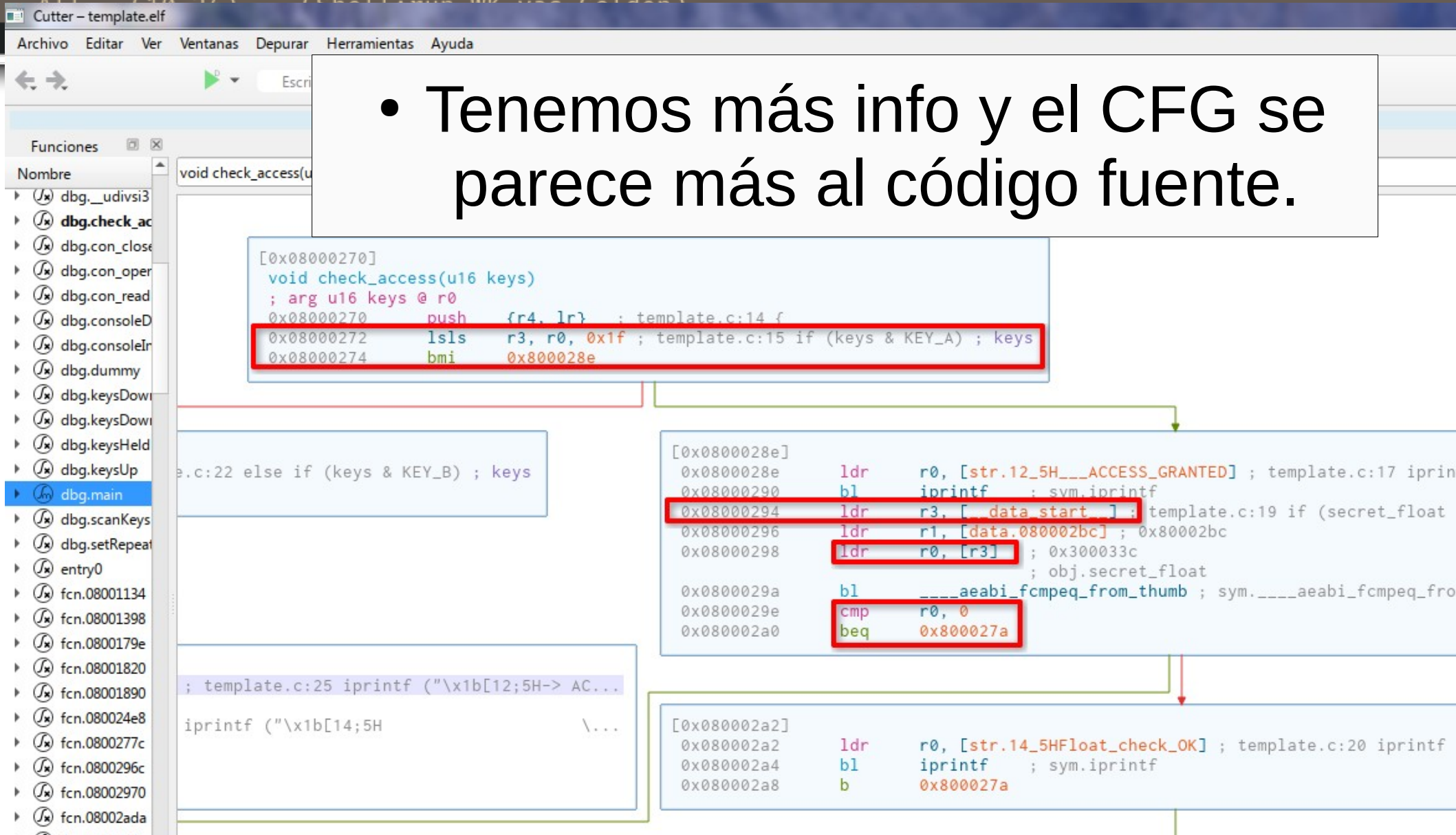
```
9 [ibai@fedora source]$ cd ../
8 [ibai@fedora template]$ make
7 make[1]: '/home/ibai/Documentos/Wiki-gba-experiment/ejemplos_gba/template/template.gba' est
6 [ibai@fedora template]$ file template.elf
5 template.elf: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked, wi
4 [ibai@fedora template]$ file template.gba
3 template.gba: Game Boy Advance ROM image: "template" (000000, Rev.00)
2 [ibai@fedora template]$ file template.sav
1 template.sav: empty
10 [ibai@fedora template]$
```

U:**- *shell* All (10,24) (Shell:run WK yas Golden)


```
template]$ file template.elf
32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked, with debug_info, not s
template]$ file template.gba
Game Boy Advance ROM image: "template" (000000, Rev.00)
template]$ file template.sav
ty
template]$
```

con debuginfo...

- Tenemos más info y el CFG se parece más al código fuente.



En esta dirección...

The screenshot shows the ImHex application interface. The main window is divided into several panes:

- Editor hexadecimal:** Displays a hex dump of a file named 'template.elf'. A white callout box with the text '¡¡Aquí!!' points to the address 0000E340, where the hex value '3C 1E 3C FF' is highlighted.
- Inspector de Datos:** A table showing various data types and their values. The 'float' type is highlighted with a value of 3.14.
- Editor de patterns:** Contains settings for pattern matching, including 'Todos los Datos', 'Selección', and 'Región'.
- Definiciones de términos de Wikipedia:** A section titled '% Decodificador de Puntos Flotantes IEEE 754' explaining the IEEE 754 standard.
- IEEE 754 Converter:** A detailed view of the IEEE 754 floating-point representation. It shows the sign, exponent, and mantissa fields, along with the decimal value 1.56234372.

At the bottom of the interface, there is a table showing the structure of the data:

Endr	Phdr	Shdr
0x00000010	0x00000033	0x00000033
0x00000034	0x00000003	0x00000003
0x0001DBB4	0x0001E26B	0x0001E26B

Below the table, the size of each section is listed: 36 bytes for Endr, 160 bytes for Phdr, and 1720 bytes for Shdr.

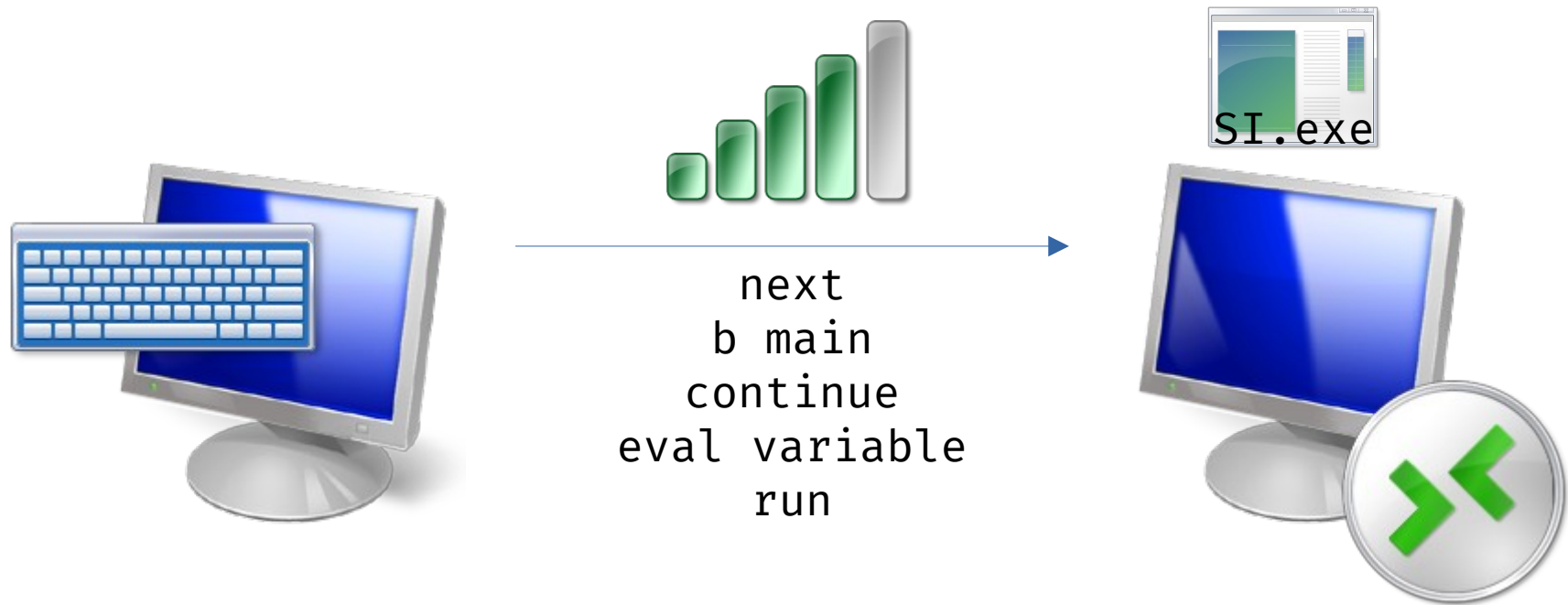
At the bottom left, the address range '0xE33C - 0xE33F (0x04 | 4 bytes)' is displayed.

- ¡¡Aquí!!
- Conversor de IEEE 754 de ImHex
- *Little endian* → al revés

Depuración

- Muchos emuladores → GDB Stub.
- Depuración remota.
 - Se utiliza más de la cuenta...
 - Abren un puerto de red.
 - Ejemplo: Depurar java con maven abre 5005.
 - Ejemplo: Depurar una consola 3ds comercial desde nuestro PC

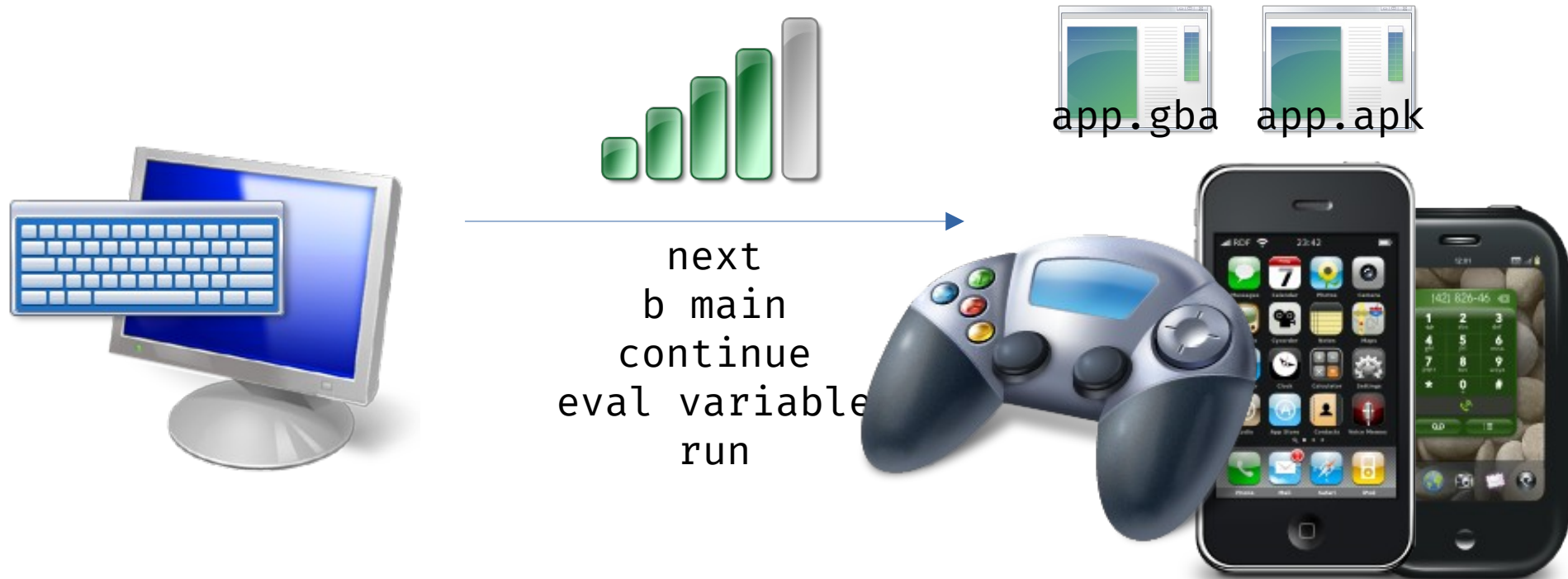
Esto nunca ha pasado



- PC en la red.
- Cliente.
- Intel gráfica.

- PC sin pantalla.
- Servidor.
- Nvidia CUDA.

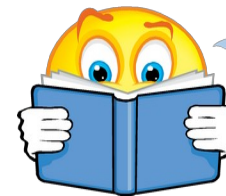
Esto nunca ha pasado



- PC en la red.
- Cliente.
- IDE de desarrollo.

- Consolas
Homebrew
- Teléfonos móvil

FIN



A quien le interese conocer más, consulta las referencias del artículo.